

1.什么是Channel

channel是.NetCore3.0之后微软借鉴golang设计思想推出的高性能生产者-消费者模型，已经在gRPC、singalR中得到了广泛的应用，其位于System.Threading.Channels中，源代码链接为：<https://github.com/dotnet/dotnet/tree/54d43b1e589bac03839771891d81b9371aa86b91/src/runtime/src/libraries/System.Threading.Channels/src/System/Threading/Channels>，其重要特性如下：

1.1 Channel的“无锁”(Lock Free)

1.首先必须明确的是channel是否“无锁”取决于用户的配置，如果使用的SPSC模型，即只有一个生产者和消费者时，是无锁设计，这时其底层本质是一个基于环状数组（Ring Buffer）和Interlocked的无锁队列，

```
1 SingleReader = true
2 SingleWriter = true
```

2.如果是多线程读写，则其内部会使用锁来保证数据的一致性，但该锁是经过优化的，只会保护用来改变内部链表指针状态的代码，性能很高

```
1 MultiReader = true
2 MultiWriter = true
```

1.2 Channel的异步特性

和BlockingCollection不一样的是，Channel的重要特性之一是它是异步的，是可await的，它不会发生线程阻塞，消费者读取数据的代码类似如下：

```
1 await channel.Reader.ReadAsync()
```

如果当前队列的数据为空，不会阻塞当前线程，而是会将后续的消费者代码打包成一个回调(Continuation)，同时释放消费者线程，当生产者再放入数据时，生产者会将数据放入队

列，同时在线程池里面派一个新的线程去执行之前的回调，放入数据的代码类似如下：

```
1 await channel.Writer.WriteAsync()
```

并且需要注意的是ReadAsync返回的是ValueTask<T>，不是Task<T>，这意味着不能多次await结果，但是由于是在stack栈上的数据，性能更高。

(1) 当队列有数据时，直接使用结构体ValueTask<T>返回数据，可实现零拷贝(Zero-Allocation)；

(2) 队列为空需要await，ValueTask无法长期挂起，它的T必须得是一个位于堆上的对象来存放状态机，Channel底层实现了IValueTaskSource<T>接口，并且加入了异步对象包装的对象缓存池机制，来最大程度避免GC，这时候拿到对象本质其实是：

ValueTask<IValueTaskSource>，IValueTaskSource接口的定义如下：

```
1 // Licensed to the .NET Foundation under one or more agreements.
2 // The .NET Foundation licenses this file to you under the MIT license.
3
4 namespace System.Threading.Tasks.Sources
5 {
6     /// <summary>Represents an object that can be wrapped by a <see
7     cref="ValueTask{TResult}">./summary>
8     /// <typeparam name="TResult">Specifies the type of data returned from the
9     object.</typeparam>
10    public interface IValueTaskSource<out TResult>
11    {
12        /// <summary>Gets the status of the current operation.</summary>
13        /// <param name="token">Opaque value that was provided to the <see
14        cref="ValueTask"/>'s constructor.</param>
15        ValueTaskSourceStatus GetStatus(short token);
16
17        /// <summary>Schedules the continuation action for this <see
18        cref="IValueTaskSource{TResult}">./summary>
19        /// <param name="continuation">The continuation to invoke when the
20        operation has completed.</param>
21        /// <param name="state">The state object to pass to <paramref
22        name="continuation"/> when it's invoked.</param>
```

```

17      /// <param name="token">Opaque value that was provided to the <see
    cref="ValueTask"/>'s constructor.</param>
18      /// <param name="flags">The flags describing the behavior of the
    continuation.</param>
19      void OnCompleted(Action<object?> continuation, object? state, short
    token, ValueTaskSourceOnCompletedFlags flags);
20
21      /// <summary>Gets the result of the <see
    cref="IValueTaskSource{TResult}"/>.</summary>
22      /// <param name="token">Opaque value that was provided to the <see
    cref="ValueTask"/>'s constructor.</param>
23      TResult GetResult(short token);
24  }
25 }

```

由于是缓存机制，因此IValueTaskSource本质是复用的，为了防止出现ABA问题，在OnCompleted方法中的token参数就是底层维护的版本号，当ValueTask创建时，它会记住当前的token，当使用GetResult获取结果时，会进行token检验。如果想自己在项目中实现一个支持零分配重用的组件，可以使用ManualResetValueTaskSourceCore来实现。

1.3 背压机制

使用Channel时可以设置队列的容量Capacity，当队列数据满了之后，可以设置为wait，此时再往队列写数据时，会挂起，避免像传统的List一样出现OOM问题。

2.ChannelOptions

```

1 // Licensed to the .NET Foundation under one or more agreements.
2 // The .NET Foundation licenses this file to you under the MIT license.
3
4 namespace System.Threading.Channels
5 {
6     /// <summary>Provides static methods for creating channels.</summary>
7     public static partial class Channel
8     {
9         /// <summary>Creates an unbounded channel usable by any number of
            readers and writers concurrently.</summary>
10        /// <returns>The created channel.</returns>

```

```

11     public static Channel<T> CreateUnbounded<T>() =>
12         new UnboundedChannel<T>(runContinuationsAsynchronously: true);
13
14     /// <summary>Creates an unbounded channel subject to the provided
options.</summary>
15     /// <typeparam name="T">Specifies the type of data in the channel.
</typeparam>
16     /// <param name="options">Options that guide the behavior of the
channel.</param>
17     /// <returns>The created channel.</returns>
18     /// <exception cref="ArgumentNullException"><paramref name="options"/>
is <see langword="null"/>.</exception>
19     public static Channel<T> CreateUnbounded<T>(UnboundedChannelOptions
options)
20     {
21         ArgumentNullException.ThrowIfNull(options);
22
23         if (options.SingleReader)
24         {
25             return new SingleConsumerUnboundedChannel<T>
(!options.AllowSynchronousContinuations);
26         }
27
28         return new UnboundedChannel<T>
(!options.AllowSynchronousContinuations);
29     }
30
31     /// <summary>Creates a channel with the specified maximum capacity.
</summary>
32     /// <typeparam name="T">Specifies the type of data in the channel.
</typeparam>
33     /// <param name="capacity">The maximum number of items the channel may
store.</param>
34     /// <returns>The created channel.</returns>
35     /// <remarks>
36     /// Channels created with this method apply the <see
cref="BoundedChannelFullMode.Wait"/>
37     /// behavior and prohibit continuations from running synchronously.
38     /// </remarks>
39     /// <exception cref="ArgumentOutOfRangeException"><paramref
name="capacity"/> is negative.</exception>
40     public static Channel<T> CreateBounded<T>(int capacity) =>
41         capacity > 0 ? new BoundedChannel<T>(capacity,
BoundedChannelFullMode.Wait, runContinuationsAsynchronously: true, itemDropped:

```

```

null) :
42         capacity == 0 ? new RendezvousChannel<T>
(BoundedChannelFullMode.Wait, runContinuationsAsynchronously: true, itemDropped:
    null) :
43         throw new ArgumentOutOfRangeException(nameof(capacity));
44
45         /// <summary>Creates a channel subject to the provided options.
</summary>
46         /// <typeparam name="T">Specifies the type of data in the channel.
</typeparam>
47         /// <param name="options">Options that guide the behavior of the
channel.</param>
48         /// <returns>The created channel.</returns>
49         /// <exception cref="ArgumentNullException"><paramref name="options"/>
is <see langword="null"/>.</exception>
50         public static Channel<T> CreateBounded<T>(BoundedChannelOptions options)
=>
51         CreateBounded<T>(options, itemDropped: null);
52
53         /// <summary>Creates a channel subject to the provided options.
</summary>
54         /// <typeparam name="T">Specifies the type of data in the channel.
</typeparam>
55         /// <param name="options">Options that guide the behavior of the
channel.</param>
56         /// <param name="itemDropped">Delegate that will be called when item is
being dropped from channel. See <see cref="BoundedChannelFullMode"/>.</param>
57         /// <returns>The created channel.</returns>
58         /// <exception cref="ArgumentNullException"><paramref name="options"/>
is <see langword="null"/>.</exception>
59         public static Channel<T> CreateBounded<T>(BoundedChannelOptions options,
Action<T>? itemDropped)
60         {
61             ArgumentNullException.ThrowIfNull(options);
62
63             return
64                 options.Capacity > 0 ? new BoundedChannel<T>(options.Capacity,
options.FullMode, !options.AllowSynchronousContinuations, itemDropped) :
65                 new RendezvousChannel<T>(options.FullMode,
!options.AllowSynchronousContinuations, itemDropped);
66         }
67     }
68 }

```

2.1 ChannelOptions

ChannelOptions是用来创建ChannelOptions相关的基类，其定义如下：

```
1 // Licensed to the .NET Foundation under one or more agreements.
2 // The .NET Foundation licenses this file to you under the MIT license.
3
4 namespace System.Threading.Channels
5 {
6     /// <summary>Provides options that control the behavior of channel
7     instances.</summary>
8     public abstract class ChannelOptions
9     {
10         /// <summary>
11         /// <code>true</code> if writers to the channel guarantee that there
12         will only ever be at most one write operation
13         /// at a time; <code>false</code> if no such constraint is guaranteed.
14         /// </summary>
15         /// <remarks>
16         /// If true, the channel may be able to optimize certain operations
17         based on knowing about the single-writer guarantee.
18         /// The default is false.
19         /// </remarks>
20         public bool SingleWriter { get; set; }
21
22         /// <summary>
23         /// <code>true</code> if readers from the channel guarantee that there
24         will only ever be at most one read operation
25         /// at a time; <code>false</code> if no such constraint is guaranteed.
26         /// </summary>
27         /// <remarks>
28         /// If true, the channel may be able to optimize certain operations
29         based on knowing about the single-reader guarantee.
30         /// The default is false.
31         /// </remarks>
32         public bool SingleReader { get; set; }
33
34         /// <summary>
35         /// <code>true</code> if operations performed on a channel may
36         synchronously invoke continuations subscribed to
```

```

31      /// notifications of pending async operations; <code>>false</code> if
all continuations should be invoked asynchronously.
32      /// </summary>
33      /// <remarks>
34      /// Setting this option to <code>>true</code> can provide measurable
throughput improvements by avoiding
35      /// scheduling additional work items. However, it may come at the cost
of reduced parallelism, as for example a producer
36      /// may then be the one to execute work associated with a consumer, and
if not done thoughtfully, this can lead
37      /// to unexpected interactions. The default is false.
38      /// 是否允许同步延续
39      /// </remarks>
40      public bool AllowSynchronousContinuations { get; set; }
41  }
42 }

```

AllowSynchronousContinuations参数：设置后续代码的执行方式是异步唤醒，还是直接在生产者自己的线程里直接执行后续任务，可以避免线程池的上下文切换开销，**但一般来说只有当后续任务是极度轻量的且不包含任何IO和耗时操作时，才将其设置为true，否则可能会卡死生产者。**

2.2 UnboundedChannelOptions

UnboundedChannelOptions用来创建没有容量上限的Channel，其定义如下：

```

1 public sealed class UnboundedChannelOptions : ChannelOptions
2 {
3 }

```

一般用于消费者速度大于生产者或者队列的数据结构很简单的情况，要注意其可能会引起OOM，但当其为SPSC时，性能也最好。

2.3 BoundedChannelOptions

BoundedChannelOptions用来创建指定容量的Channel，其定义如下：

```

1 public sealed class BoundedChannelOptions : ChannelOptions
2 {
3     /// <summary>The maximum number of items the bounded channel may store.
4     </summary>
5     private int _capacity;
6     /// <summary>The behavior incurred by write operations when the channel is
7     full.</summary>
8     private BoundedChannelFullMode _mode = BoundedChannelFullMode.Wait;
9
10    /// <summary>Initializes the options.</summary>
11    /// <param name="capacity">The maximum number of items the bounded channel
12    may store.</param>
13    /// <exception cref="ArgumentOutOfRangeException"><paramref
14    name="capacity"/> is negative.</exception>
15    public BoundedChannelOptions(int capacity)
16    {
17        if (capacity < 0)
18        {
19            throw new ArgumentOutOfRangeException(nameof(capacity));
20        }
21
22        _capacity = capacity;
23    }
24
25    /// <summary>Gets or sets the maximum number of items the bounded channel
26    may store.</summary>
27    /// <exception cref="ArgumentOutOfRangeException"><paramref name="value"/>
28    is negative.</exception>
29    public int Capacity
30    {
31        get => _capacity;
32        set
33        {
34            if (value < 0)
35            {
36                throw new ArgumentOutOfRangeException(nameof(value));
37            }
38            _capacity = value;
39        }
40    }
41 }

```



```

36    /// <summary>Gets or sets the behavior incurred by write operations when
    the channel is full.</summary>
37    /// <exception cref="ArgumentOutOfRangeException"><paramref name="value"/>
    is an invalid enum value.</exception>
38    public BoundedChannelFullMode FullMode
39    {
40        get => _mode;
41        set
42        {
43            switch (value)
44            {
45                case BoundedChannelFullMode.Wait:
46                case BoundedChannelFullMode.DropNewest:
47                case BoundedChannelFullMode.DropOldest:
48                case BoundedChannelFullMode.DropWrite:
49                    _mode = value;
50                    break;
51                default:
52                    throw new ArgumentOutOfRangeException(nameof(value));
53            }
54        }
55    }

```

- Capacity参数：设置容量上限；
- BoundedChannelFullMode参数：设置容量满了之后的策略，有三种
 - Wait：生产者的WriteAsync会挂起，一直等待可以有数据写入，不会丢失数据，但是会拖慢生产者的速度（背压机制）
 - DropNewest：并不是丢弃最新的数据，而是将队列尾部刚刚加入的数据丢弃，即前一个数据丢弃，并将当前数据写入，所以保留的是最早进入和最后进入的数据，中间的数据被丢弃
 - DropOldest：丢弃最旧的数据
 - DropWrite：将当前即将写入的数据丢弃，用于保护当前队列里的数据不受影响

2.4 UnboundedPrioritizedChannelOptions

UnboundedPrioritizedChannelOptions用来创建无容量的，具有优先级的Channel，其定义如下：

```
1 public sealed class UnboundedPrioritizedChannelOptions<T> : ChannelOptions
2 {
3     public UnboundedPrioritizedChannelOptions();
4
5     //
6     // Summary:
7     //     Gets or sets the comparer used by the channel to prioritize elements.
8     public IComparer<T>? Comparer { get; set; }
9 }
```

UnboundedPrioritizedChannelOptions不在是FIFO，而是根据IComparer的结果决定谁先出队，如果不指定Compare的话，则会调用默认的IComparer，值越小，越小出队列，其底层不再是Ring Buffer，而是一个并发安全的Min-Heap，但对应的是其吞吐量会下降，因为队列的复杂度是O(1)，但Min-Heap则是O(logN)，且会出现线程饥饿问题。

2.5 最佳实践

如果不能100%保证消费者的速度大于生产者，一般优先使用BoundedChannelOptions，尤其是对于HTTP请求，当队列满了之后可以DropWrite，并返回503来保护服务的稳定性。

3.Channel的生产消费模式及生命周期控制

根据SingleWriter和SingleReader的配置，可以组合出四种生产-消费者模式。

3.1 SPSC模式 (SingleWriter-SingleReader)

```
1 public async Task RunOneToOne()
2 {
3
4     var options = new UnboundedChannelOptions()
5     {
```

```
6         SingleReader = true,
7
8         SingleWriter = true,
9     };
10
11     var channel = Channel.CreateUnbounded<int>(options);
12
13     // producer
14
15     var producer = Task.Run(async () => {
16
17         for(int i = 0; i < 10; i++)
18         {
19
20             await channel.Writer.WriteAsync(i);
21             Console.WriteLine($"{DateTime.Now}:Producer:{i}");
22             await Task.Delay(100);
23         }
24
25         channel.Writer.Complete();
26
27     });
28
29     // consumer
30
31     var consumer = Task.Run(async () =>
32     {
33
34         await foreach (var data in channel.Reader.ReadAllAsync())
35         {
36             Console.WriteLine($"{DateTime.Now}:Consumer:{data}");
37         }
38
39     });
40
41     await producer;
42
43     await consumer;
```

```
45     Console.WriteLine("Finished");
46
47 }
```

需要注意的是我们得使用`channel.Writer.Complete()`告诉消费者数据写入结束，而不能建议用`CancellationToken`传入`channel.Reader.ReadAllAsync()`，这样会造成队列中剩下的数据未处理完，产生丢失。

3.2 1:N模式 (SingleWriter–MultiReader)

```
1 public async Task RunOneToN()
2 {
3     var options = new UnboundedChannelOptions()
4     {
5         SingleReader = false,
6
7         SingleWriter = true,
8     };
9
10    var channel = Channel.CreateUnbounded<int>(options);
11
12    // producer
13
14    var producer = Task.Run(async () => {
15
16        for (int i = 0; i < 20; i++)
17        {
18
19            await channel.Writer.WriteAsync(i);
20            Console.WriteLine($"{DateTime.Now}:Producer:{i}");
21            await Task.Delay(100);
22        }
23
24        channel.Writer.Complete();
25
26    });
27 }
```

```

28     List<Task> consumers = [];
29
30     const int n = 5;
31
32     for (int i = 0; i < n; i++)
33     {
34         int lockcalI = i;
35
36         consumers.Add(Task.Run(async () =>
37         {
38
39             await foreach (var data in channel.Reader.ReadAllAsync())
40             {
41
42
43                 Console.WriteLine($"{DateTime.Now}:consumerId:
44 {lockcalI}:Consumer:{data}");
45
46             }
47         }));
48     }
49
50     await producer;
51
52     await Task.WhenAll(consumers);
53
54     Console.WriteLine("Finished");
55
56 }

```

3.3 N:1模式 (MultiWriter–SingleReader)

```

1  public async Task RunNToOne()
2  {
3
4      var options = new UnboundedChannelOptions()

```

```
5      {
6          SingleReader = true,
7
8          SingleWriter = false,
9      };
10
11      var channel = Channel.CreateUnbounded<string>(options);
12
13      // producer
14      const int n = 10;
15
16      List<Task> producers = [];
17
18      for (int i = 0; i < n; i++)
19      {
20
21          int localI = i;
22
23          producers.Add(Task.Run(async () =>
24              {
25
26                  await channel.Writer.WriteAsync($"ProducerId:{localI}, Producer:
27                  {localI}");
28
29              }));
30
31      }
32
33      // consumer
34      var consumer = Task.Run(async () =>
35      {
36
37          await foreach(var data in channel.Reader.ReadAllAsync())
38              {
39
40                  Console.WriteLine($"{DateTime.Now}:{data}");
41
42              }
43
44      });
```

```

44     });
45
46     await Task.WhenAll(producers).ContinueWith(_ =>
47     {
48         channel.Writer.Complete();
49     });
50
51     await consumer;
52
53     Console.WriteLine("Finished");
54
55 }

```

需要注意的是必须等生产者任务结束之后使用channel.Writer.Complete()告诉消费者数据写入结束。

3.3 N:N模式 (MultiWriter-MultiReader)

```

1  public async Task RunNTonN()
2  {
3
4      var options = new UnboundedChannelOptions()
5      {
6          SingleReader = false,
7
8          SingleWriter = false,
9      };
10
11     var channel = Channel.CreateUnbounded<string>(options);
12
13     // producer
14     var producer = Enumerable.Range(0, 5).Select(i => Task.Run(async () =>
15     {
16
17         for (int j = 0; j < 4; j++)
18         {
19

```

```

20         await channel.Writer.WriteAsync($"ProcuderId:{i}, {i * 10 + j}");
21
22     }
23
24     })).ToList();
25
26     // consumer
27
28     var consumers = Enumerable.Range(0, 3).Select(i => Task.Run(async () =>
29     {
30
31         await foreach(var data in channel.Reader.ReadAllAsync())
32         {
33
34             Console.WriteLine($"ConsumerId:{i}, data:{data}");
35
36             await Task.Delay(50);
37
38         }
39
40     }));
41
42     await Task.WhenAll(producer).ContinueWith(_ =>
43     {
44         channel.Writer.Complete();
45     });
46
47     await Task.WhenAll(consumers);
48
49     Console.WriteLine("Finished");
50
51 }

```

4.Rag上传文档案例

```

1 public interface IWorkingChannel<T>
2 {

```



```
3
4     ValueTask EnqueueJobAsync(T job, CancellationToken cancellationToken =
    default);
5
6     IEnumerable<T> DequeueJobsAsync();
7
8     void Stop();
9 }
```

```
1 public abstract class WorkingChannelBase<T> : IWorkingChannel<T>
2 {
3
4     protected Channel<T> _channel = null;
5
6
7     public async ValueTask EnqueueJobAsync(T job, CancellationToken
    cancellationToken = default)
8     {
9         ArgumentNullException.ThrowIfNull(job);
10        await _channel.Writer.WriteAsync(job, cancellationToken);
11    }
12
13    public IEnumerable<T> DequeueJobsAsync()
14    {
15        // 所有的取消操作由 Complete 来完成，不需要传入 token
16        return _channel.Reader.ReadAllAsync();
17    }
18
19
20    public void Stop()
21    {
22        _channel.Writer.Complete();
23    }
24
25
26 }
```

```
1 public class StandardWorkingChannel<T> : WorkingChannelBase<T>
2 {
3
4
5     public StandardWorkingChannel(UnboundedChannelOptions options)
6     {
7
8         _channel = Channel.CreateUnbounded<T>(options);
9
10    }
11
12    public StandardWorkingChannel(BoundedChannelOptions options)
13    {
14
15        _channel = Channel.CreateBounded<T>(options);
16
17    }
18
19
20 }
```

```
1 public class PrioritizedWorkingChannel<T> : WorkingChannelBase<T> where T :
   IComparable<T>
2 {
3     public PrioritizedWorkingChannel(UnboundedPrioritizedChannelOptions<T>
   options)
4     {
5         _channel = Channel.CreateUnboundedPrioritized<T>(options);
6     }
7 }
```

```
1 public class DocumentImportJob(string filePath, string documentId, string
  documentName, int categoryId, WorkPriority priority = WorkPriority.VIP) :
  IComparable<DocumentImportJob>
2 {
3
4     public string FilePath { get; set; } = filePath;
5
6     public string DocumentId { get; set; } = documentId;
7
8     public string DocumentName { get; set; } = documentName;
9
10    public int CategoryId { get; set; } = categoryId;
11
12    public DateTime EnqueueTime { get; set; } = DateTime.UtcNow;
13
14    public WorkPriority Priority { get; set; } = priority;
15
16
17    private DateTime GetScore()
18    {
19
20        return Priority switch
21        {
22
23            WorkPriority.VIP => EnqueueTime,
24
25            WorkPriority.High => EnqueueTime.AddMinutes(10),
26
27            WorkPriority.Normal => EnqueueTime.AddMinutes(30),
28
29            _ => EnqueueTime.AddMinutes(60),
30
31        };
32
33    }
34
35
36    public int CompareTo(DocumentImportJob? other)
```

```
37     {
38         return other == null ? 1 : GetScore().CompareTo(other.GetScore());
39     }
40 }
41
42
43 }
```

关键点：

- 使用面向接口编程和继承，避免重复的代码和泛型约束污染，同时方便在ASP.NetCore中进行依赖注入，代码如下：

```
1 builder.Services.AddSingleton<IWorkingChannel<DocumentImportJob>>(sp=>new
   PrioritizedWorkingChannel<DocumentImportJob>(
2     new
   System.Threading.Channels.UnboundedPrioritizedChannelOptions<DocumentImportJob>
   { SingleReader = true, SingleWriter = false}));
```

- ASP.NetCore中进行依赖注入时需要注意注册成单例模式；
- 参考Linux的CFS，通过时间惩罚算法，避免线程饥饿问题
- Channel不需要Dispose，因为其为托管数据结构，没有占用操作系统底层的句柄，C#的GC已经足够